Teste de Software

Detecção de Bad Smells e Refatoração Segura

Aluno: Flavio de Souza Ferreira Junior

Matrícula: 859363

27 de outubro de 2026

1. Análise do código original

Método longo. O método `generateReport` fazia cabeçalho, filtro de itens, formatação CSV/HTML e totalização. Quando tudo fica junto, uma mudança pequena exige reler o método inteiro e aumenta a chance de afetar outra parte.

Condicionais aninhadas. A lógica repetia decisões por perfil (ADMIN/USER) e formato (CSV/HTML) dentro do mesmo laço. Isso dificulta enxergar quais itens entram no relatório e quais casos precisam ser testados.

Duplicação e números mágicos. A linha CSV e a soma do total apareciam nos dois perfis. Os limites 500 e 1000 estavam soltos no código. Se uma regra mudar, seria fácil atualizar um lugar e esquecer outro.

2. Resultado do ESLint antes da refatoração

Print da primeira execução de `npx eslint src/`, com o código original ainda sem ajustes:

```
$ npx eslint src/

/Users/flferreira/bad-smell-test/src/ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity
                from 27 to the 15 allowed  sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one
                sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

O SonarJS mediu complexidade cognitiva 27, acima do limite 15, e apontou um `if` que poderia ser simplificado. A duplicação e os números mágicos apareceram mais claramente na leitura manual. A medida de complexidade ajudou a confirmar a gravidade dos vários desvios dentro do método.

3. Processo de refatoração

Escolhi como principal problema a combinação de laço longo com muitos `if/else`. Antes, a decisão do perfil e a decisão do formato ficavam misturadas no mesmo trecho:

Antes (trecho resumido do original):

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) item.priority = true;
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') { /* ... */ }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        report += `${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
      }
    }
  }
}
```

Depois (arquivo refatorado):

```
for (const item of items) {
  if (!this.isVisibleTo(user, item)) continue;
  this.markAdminPriority(user, item);
  if (reportType === 'CSV') {
    report += this.csvRow(user, item);
    total += item.value;
  } else if (reportType === 'HTML') {
    report += this.htmlRow(user, item);
    total += item.value;
  }
}
```

Usei **Extract Method** para separar filtro, prioridade, cabeçalho, linhas e rodapé. Assim a regra de visibilidade fica em um lugar só, e a formatação de cada tipo também. Troquei 500 e 1000 por constantes com nomes. A complexidade cognitiva do método principal caiu de 27 na primeira medição para 6.

4. Validação

No projeto-base, o Jest precisava da opção para ES Modules. Depois de habilitá-la, 4 dos 5 testes originais passaram. O teste HTML de ADMIN revelou um defeito já existente: o código escrevia `<tr >` em vez de `<tr>`. Fiz essa correção mínima no arquivo original, sem mudar a suíte de testes.

A suíte original ficou em `tests/ReportGenerator.test.js`, sem alterações. A cópia em `__tests__/ReportGenerator.refactored.test.js` mudou somente o import para testar o novo arquivo.

Resultado final: 2 suítes e 10 testes passaram; `npx eslint src/ReportGenerator.refactored.js` terminou sem erros. Comparei ainda 27 cenários entre as duas versões, incluindo perfis, formatos, limites e itens vazios: a saída e os efeitos sobre os itens foram iguais.

5. Conclusão

Os testes ajudaram de verdade: primeiro mostraram um erro que já estava no projeto e, depois, deram segurança para reorganizar o código. O método menor e as regras separadas tornam mais fácil entender onde mexer quando surgir um novo formato ou uma nova regra de usuário. O ESLint complementou a leitura manual ao mostrar a complexidade de forma objetiva.

Repositório do trabalho: github.com/flaviojuniordev/bad-smell-test

Projeto-base: github.com/CleitonSilvaT/bad-smells-js-refactoring (commit 39d39f9)